

UNIT 2 – DATA STRUCTURES AND COLLECTION MANAGEMENT

Smart Library Management System

Student Name	Srinivas
PRN	2126UDSM1108
Department	AI-DS
Year / Semester	1 / 1

1. Problem Analysis

The Smart Library Management System organizes library book information using Python data structures. The system should store book details, search and update records, manage categories, count available books, find books by author, sort records, and delete books when required.

Requirement	Description
Book storage	Store complete information for each book.
Search	Search books by title and author.
Update	Modify book details when required.
Categories	Maintain unique categories without duplicates.
Availability	Count books that are currently available.
Sorting	Arrange books alphabetically.
Deletion	Remove a selected book record.

2. Data Structure Selection

Data Structure	Purpose
String	Store text such as title, author, publisher and category.
List	Store multiple book records in an ordered collection.
Tuple	Store fixed groups of related values.
Set	Store unique categories and remove duplicates.
Dictionary	Store key-value pairs for each book record.

3. Algorithms

Add New Book

Start → Read book details → Create a dictionary record → Add the record to the library list → Display confirmation → Stop

Search by Title

Start → Read the required title → Check each book in the library → Compare the title → Display matching record(s) → Stop

Update Book

Start → Read book ID → Search for the matching record → Update required fields → Display confirmation → Stop

Remove Duplicate Categories

Start → Collect category values → Convert them to a set → Display unique categories → Stop

Count Available Books

Start → Set count to zero → Check every book → If available, increase count → Display count → Stop

Find Books by Author

Start → Read author name → Check each book → Display books with matching author → Stop

Sort Books Alphabetically

Start → Sort library records using title → Display sorted records → Stop

Delete Book

Start → Read book ID → Find matching record → Remove the record → Display confirmation → Stop

4. Flowchart Logic

The program follows the sequence: Start → Input data → Select required operation → Process the selected data structure → Display result → End. For search, update and delete operations, the program checks whether the requested record exists before performing the operation.

5. Pseudocode

```
BEGIN
    Create empty library list

    ADD BOOK:
        Read book details
        Create book dictionary
        Add dictionary to library

    SEARCH:
        Read title
        For each book in library
            If title matches
                Display book

    UPDATE:
        Read book ID
        Find matching book
        Update required details

    UNIQUE CATEGORIES:
        Collect categories
        Convert categories to a set
        Display unique categories

    COUNT AVAILABLE:
        Set count = 0
        For each book
            If available is True
                count = count + 1
        Display count

    FIND BY AUTHOR:
        Read author
        Display matching books

    SORT:
        Sort books by title
        Display books

    DELETE:
        Read book ID
        Remove matching book

END
```

6. Python Implementation

```
library = []

def add_book(book):
    library.append(book)

def unique_categories():
    categories = {book["category"] for book in library}
    return categories

def find_by_title(title):
    return [book for book in library
            if book["title"].lower() == title.lower()]

def update_book(book_id, **updates):
    for book in library:
        if book["book_id"] == book_id:
            book.update(updates)
            return True
    return False

def count_available():
    return sum(book["available"] for book in library)
```

```

def find_by_author(author):
    return [book for book in library
            if book["author"].lower() == author.lower()]

def sort_books():
    return sorted(library, key=lambda book: book["title"].lower())

def delete_book(book_id):
    for book in library:
        if book["book_id"] == book_id:
            library.remove(book)
            return True
    return False

# Sample records
add_book({
    "book_id": "B1042",
    "title": "Python Programming",
    "author": "John Smith",
    "publisher": "Tech Books",
    "category": "Programming",
    "available": True,
    "price": 450,
    "shelf_number": "A12"
})

add_book({
    "book_id": "B1043",
    "title": "Data Structures",
    "author": "Jane Doe",
    "publisher": "Academic Press",
    "category": "Programming",
    "available": True,
    "price": 520,
    "shelf_number": "A13"
})

print("Books:", library)
print("Unique categories:", unique_categories())
print("Search result:", find_by_title("Python Programming"))
print("Available books:", count_available())
print("Author search:", find_by_author("John Smith"))
print("Sorted books:", sort_books())

```

7. Data Structure Practice Programs

String

```

title = "Python Programming"
print(title.upper())
print(title.lower())

```

List

```

books = ["Python", "Java", "C"]
books.append("C++")
print(books)

```

Tuple

```

book_info = ("B1042", "Python Programming", "John Smith")
print(book_info)

```

Set

```

categories = {"Programming", "Science", "Programming"}
print(categories)

```

Dictionary

```

book = {"book_id": "B1042", "title": "Python Programming", "available": True}
print(book["title"])

```

8. Testing Report

Testing checks whether the program produces the expected result for normal, boundary, invalid, duplicate and case-sensitive inputs.

Test Case	Input / Condition	Expected Result
Normal	Search for an existing title	Matching book is displayed
Boundary	Library contains one record	Record is processed correctly
Invalid	Search for a non-existing title	No matching record is found
Duplicate	Repeated category values	Set removes duplicates
Case sensitivity	Different letter case in search	Case-insensitive match works

9. Trace Table

Step	Operation	Result
1	Add B1042	Record added
2	Add B1043	Record added
3	Find unique categories	Programming
4	Count available books	2
5	Sort by title	Data Structures, Python Programming

10. Documentation

The project documentation explains the problem, selected data structures, algorithms, flowchart logic, pseudocode, Python implementation, testing process and results. The final submission should contain the required programs and reports in the format specified by the assignment.

11. Submission Checklist

- Problem analysis completed
- Data structure selection completed
- Algorithms prepared
- Flowchart logic prepared
- Pseudocode prepared
- Python programs completed
- Integrated program completed
- Testing report completed
- Trace table completed
- Documentation completed

Student: Srinivas | PRN: 2126UDSM1108